

Relatório Técnico — Parte Prática

Projeto “Pedidos Veloz” — implementação fim a fim

Caso: Loja Veloz | Stack: Python/FastAPI, Docker, Kubernetes, GitHub Actions

Aluno: Bruno Maximino dos Reis

Vídeo pitch: [INSERIR LINK DO YOUTUBE]

1. Visão geral e arquitetura

A solução implementa a plataforma “Pedidos Veloz” como quatro microsserviços de domínio, um banco PostgreSQL e mensageria RabbitMQ. O **API Gateway** é o único ponto de entrada HTTP; ele encaminha as requisições ao serviço de **Pedidos**, que orquestra a reserva no **Estoque**, a cobrança no serviço de **Pagamentos** (integração externa simulada), persiste o pedido e publica o evento assíncrono “**PedidoCriado**” na fila.

Cliente ■■■■ API Gateway ■■■■ Orders ■■■■■■ Inventory
■■■■ Payments (gateway externo)
■■■■ PostgreSQL (persistência)
■■■■ RabbitMQ (evento PedidoCriado)

Serviço	Responsabilidade	Porta / Exposição
api-gateway	Entrada HTTP e roteamento	8080 (externo)
orders	Cria/consulta pedidos, orquestra, emite evento	interno
payments	Cobrança (integração externa simulada)	interno
inventory	Reserva e baixa de estoque	interno
db / rabbitmq	Persistência / mensageria de eventos	interno

2. Ambiente local com Docker Compose

Toda a stack sobe com um único comando. O Compose define duas redes (*frontend* e *backend*, isolando o que é exposto do que é interno), um volume persistente para o banco, variáveis de ambiente por serviço e *healthchecks* com *depends_on* condicional — o serviço de Pedidos só inicia quando o banco e a fila estão saudáveis.

```
cp .env.example .env
docker compose up --build      # sobe toda a stack

curl -X POST http://localhost:8080/api/orders \
  -H 'Content-Type: application/json' \
  -d '{"customer_id":"cli-1","item_sku":"SKU-001","quantity":2,"amount":199.90}'
```

3. Containerização e versionamento

Cada serviço tem um Dockerfile **multi-stage**: o primeiro estágio instala as dependências; o segundo copia apenas o necessário para uma imagem final enxuta. Boas práticas de segurança aplicadas: usuário **não-root**, redução de camadas, dependências mínimas (*python:3.12-slim*) e *healthcheck* na imagem. As imagens são versionadas no pipeline por *SHA* curto, *semver* (em tags) e *latest* no branch principal.

```
FROM python:3.12-slim AS builder
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install/deps -r requirements.txt

FROM python:3.12-slim
RUN useradd --uid 10001 appuser
COPY --from=builder /install/deps /deps
COPY app/ ./app/
USER appuser          # nunca executa como root
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

4. Kubernetes — produção mínima

Os manifests estão organizados por responsabilidade (namespace/config, banco, serviços, escala/rede). Cada serviço tem **Deployment** e **Service**; o PostgreSQL é um **StatefulSet** com volume persistente. A configuração não sensível vai em **ConfigMap**; credenciais (URL do banco, chave do gateway) em **Secret**, injetadas como variáveis de ambiente.

Segurança aplicada e justificativa:

- **Pod Security Admission “restricted”** no namespace: bloqueia pods privilegiados e exige boas práticas por padrão — defesa na entrada do cluster.
- **securityContext**: *runAsNonRoot*, *readOnlyRootFilesystem*, *allowPrivilegeEscalation: false* e *drop ALL* de capabilities — reduz o impacto de um contêiner comprometido.
- **NetworkPolicy default-deny** de ingresso: só o tráfego explicitamente permitido trafega — segmentação de rede.
- **readiness/liveness probes**: o cluster reinicia processos travados e só envia tráfego a réplicas prontas, evitando indisponibilidade durante deploys.

Cada serviço expõe */health/live* e */health/ready*; o readiness do serviço de Pedidos verifica a conexão com o banco, garantindo que réplicas sem banco não recebam requisições.

5. CI/CD (GitHub Actions)

O pipeline executa em duas fases. **(1) test**: em matriz para os quatro serviços, roda lint (ruff) e testes (pytest). **(2) build-and-push** (apenas em push para main/tags): faz o build da imagem, executa **scan de vulnerabilidades** com Trivy e publica no GitHub Container Registry. As credenciais usam o *GITHUB_TOKEN* gerenciado — nenhum segredo no código. O *gate* de testes impede que código quebrado seja publicado.

Etapa	Ferramenta	Gate
Lint	ruff	Informativo
Testes	pytest (14 testes)	Bloqueante
Build	docker buildx	Bloqueante
Scan	Trivy (CRITICAL/HIGH)	Informativo no MVP
Publicação	GHCR + tags SHA/semver	Só em main/tags

6. Observabilidade

Os três pilares estão contemplados. **Métricas**: todos os serviços expõem */metrics* no formato Prometheus (pedidos criados, cobranças por resultado, requisições no gateway, nível de estoque). **Logs**: emitidos em JSON estruturado para stdout (12-Factor), prontos para agregação por Loki/Fluent Bit. **Traces**: instrumentação OpenTelemetry (OTLP) enviando spans a um Collector que exporta para Jaeger/Tempo, permitindo seguir uma requisição *gateway* → *orders* → *inventory* → *payments* ponta a ponta, com o *trace_id* propagado entre os serviços.

```
export OTEL_EXPORTER_OTLP_ENDPOINT=http://otel-collector:4317
export OTEL_SERVICE_NAME=orders
opentelemetry-instrument uvicorn app.main:app --host 0.0.0.0 --port 8000
```

7. Estratégia de deploy

Adotamos **Rolling Update** como padrão (nativo do Deployment): novas réplicas sobem e passam pelo readiness antes que as antigas sejam removidas, garantindo indisponibilidade zero — a dor relatada de “quedas durante deploys”. Para o serviço de **Pedidos**, no caminho crítico da campanha, recomendamos evoluir para **Canary**: liberar a nova versão para uma fração do tráfego, observar métricas e erros e só então promover. O **Blue-Green** foi considerado, mas exige o dobro de recursos em paralelo; o Canary oferece rollback igualmente rápido com custo menor — daí a escolha.

8. Escalabilidade

Usamos **HPA (Horizontal Pod Autoscaler)** por utilização de CPU nos serviços do caminho crítico: *orders* (3→15 réplicas, alvo 65%) e *api-gateway* (2→10, alvo 70%). A política de *behavior* sobe rápido (janela de 30s) e desce devagar (300s), evitando oscilação. O **HPA** foi preferido ao **VPA** porque o gargalo esperado em campanha é volume de requisições concorrentes — resolvido somando réplicas, não engordando uma só. O VPA fica como recomendação futura para ajuste fino de *requests/limits* de serviços de carga estável (ex.: o banco), pois ambos não devem atuar sobre a mesma métrica simultaneamente.

```
minReplicas: 3
maxReplicas: 15
metrics:
  - type: Resource
    resource: { name: cpu, target: { type: Utilization, averageUtilization: 65 } }
```

9. Infraestrutura como Código (Terraform)

Um esqueleto Terraform provisiona o cluster Kubernetes gerenciado de forma **reproduzível e versionada**: providers com versão travada, variáveis por ambiente, sugestão de *state* remoto com lock e um *node pool* com autoscaling alinhado ao HPA. A justificativa do esqueleto é demonstrar a abordagem de IaC dentro do prazo de quatro semanas; a evolução natural é modularizar (*network*, *cluster*, *node_pool*) e separar *workspaces* por ambiente.

10. Reprodução e conclusão

O repositório traz README com instruções completas: subir local com *docker compose up --build*, rodar os 14 testes e aplicar os manifests no Kubernetes. A proposta endereça diretamente as dores da Loja Veloz: o **Rolling/Canary** elimina quedas no deploy; o **HPA** dá escala sob demanda no pico; o **CI/CD com testes e scan** reduz o tempo e o risco de entrega; e a **observabilidade** (métricas, logs e tracing) ataca a baixa rastreabilidade de falhas entre serviços. O resultado é um caminho claro e automatizado do desenvolvimento à produção.